

CHAPTER 5

Kinematics

5.1. Space story

One of the challenges for a momentum-bias communication is to reorient it from its spinning transfer-orbit orientation to its mission orientation. In transfer orbit, it is a spinner and the momentum wheels are fixed. In mission orbit, the momentum-wheel axis is normal to orbit normal and the spacecraft is rotating at orbit rate, that is one revolution per day in geosynchronous orbit. The obvious approach is to despin the spacecraft with thrusters, reorient it with thrusters and then acquire the Earth. This would require three-axis sensing. Dr. Carl Hubert of RCA Astro-Electronics came up with the idea of the dual-spin turn. After the burn that puts the spacecraft in orbit, you align the spin axis with orbit normal. The same sensing can be used that is used for all of the transfer orbit. You spin down to the right angular rate so that the total angular momentum is what you want in the momentum wheel. You then turn on the momentum wheel. Angular-momentum conservation ensures that the momentum-wheel vector will align itself with orbit normal. Some damping is required. A patent was filed for major-axis spinners that included damping. However, it turned out that if the spacecraft had fuel, and any kind of mechanism to damp slosh in the tanks, then fuel slosh provided sufficient damping so that no additional damping was needed. This provided a mechanism for other satellite manufacturers to use the dual-spin-turn without a patent license. Dr. Hubert later patented a version that allowed you to do the dual-spin-turn with the starting spin about the minor inertia axis.

5.2. Introduction

Given two frames, the orientation of one frame with respect to the other can be defined in several ways. Three that are widely used are quaternions, Euler angles, and transformation matrices. A quaternion is a four-element vector incorporating an axis and angle of rotation. Euler angles are a series of three rotations. A transformation matrix is an orthonormal matrix that transforms a vector measured in one frame to another frame. An orthonormal matrix is a matrix whose inverse equals its transpose and for which each column or row is a unit vector. Fig. 5.1 shows a vector that could be measured in either of two frames.

The vector orientation and length do not change, but its x , y , and z components change depending on which frame is used to measure it. Kinematics in spacecraft refers to the expression and propagation of the satellite attitude, using one of these represen-

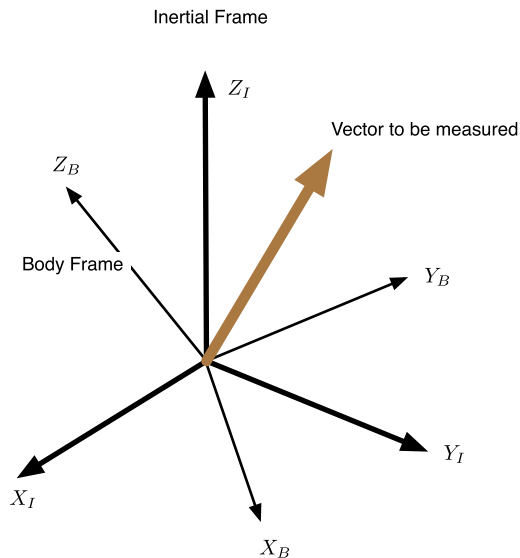


Figure 5.1 Two-frame vector diagram.

tations, without considering any accelerations. The effect of accelerations is considered in the next chapter, Dynamics.

5.3. Euler angles

Euler angles are commonly used to represent attitude or orientation. Fig. 5.2 shows a 3-2-1 Euler angle set. The first rotation is about the 3-axis of both the inertial and body frames. The second is about the 2-axis of the body frame and the third is about the 1-axis of the body frame.

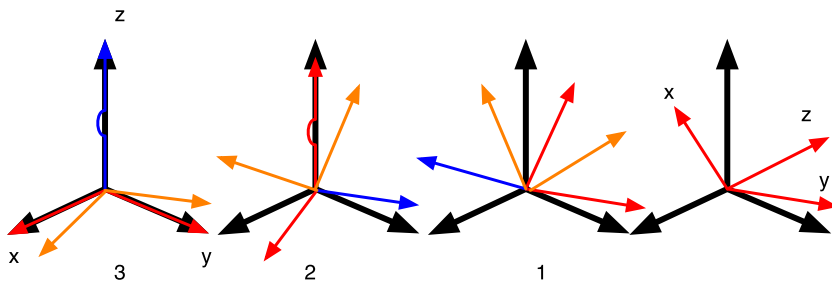


Figure 5.2 Euler-angle diagram.

Each rotation can be used to define a transformation matrix. The product of three transformation matrices gives the overall transformation matrix.

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta_1 & \sin \theta_1 \\ 0 & -\sin \theta_1 & \cos \theta_1 \end{bmatrix} \begin{bmatrix} \cos \theta_2 & 0 & -\sin \theta_2 \\ 0 & 1 & 0 \\ \sin \theta_2 & 0 & \cos \theta_2 \end{bmatrix} \begin{bmatrix} \cos \theta_3 & \sin \theta_3 & 0 \\ -\sin \theta_3 & \cos \theta_3 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (5.1)$$

There are twelve sets of Euler angles. Each set of three rotations is capable of pointing a unit vector in any orientation. Alternately, M could be obtained by a single rotation about a unit vector. The combination of rotation angle and unit vector gives the quaternion.

It is generally not recommended to propagate Euler angles. If you propagate Euler angles the propagator must compute numerous trigonometric functions, which slows down the propagation. In addition, there are singularities where only two axes of rotation are defined. It is much simpler (and computationally more efficient) to propagate quaternions (Section 5.5).

5.4. Transformation matrices

If u_b is the vector in the body frame, and u_I is the vector as measured in the inertial frame, the two are related by

$$u_I = M_{B \rightarrow I} u_b \quad (5.2)$$

Note that

$$M_{I \rightarrow B} = M_{B \rightarrow I}^T \quad (5.3)$$

The transformation matrix is defined as

$$M_{B \rightarrow I} = \begin{bmatrix} x_{B \rightarrow I}^T \\ y_{B \rightarrow I}^T \\ z_{B \rightarrow I}^T \end{bmatrix} \quad (5.4)$$

where each row is the unit vector of the axis of the body (B) frame as measured in the inertial (I) frame.

Fig. 5.3 shows two coordinate frames. Each of the axes is represented by a unit vector such that

$$Z \times X = Y \quad (5.5)$$

$$X \times Y = Z \quad (5.6)$$

$$z \times x = y \quad (5.7)$$

$$x \times y = z \quad (5.8)$$

This implies that the coordinate frames are orthogonal, i.e., every axis makes a right angle with every other axis.

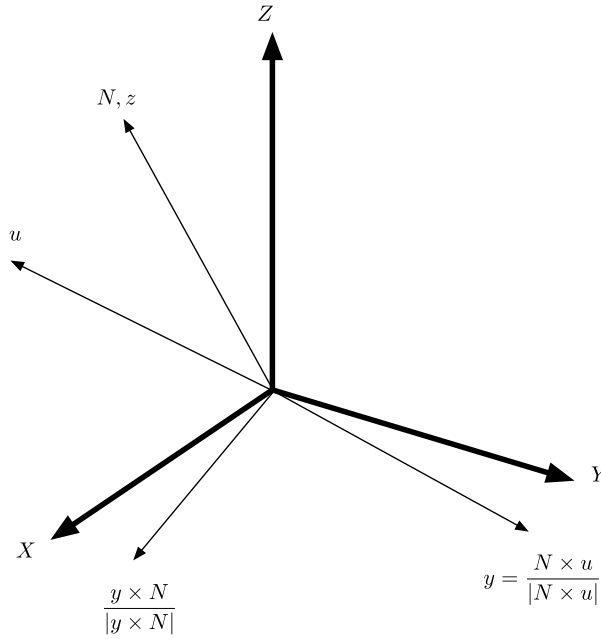


Figure 5.3 Transformation matrix from two vectors.

The vector a can be parameterized using coordinates from either set of axes. If x , y , and z are parameterized as

$$x(X, Y, Z) \quad (5.9)$$

$$y(X, Y, Z) \quad (5.10)$$

$$z(X, Y, Z) \quad (5.11)$$

then

$$a(x, y, z) = \begin{bmatrix} x(X, Y, Z) \\ y(X, Y, Z) \\ z(X, Y, Z) \end{bmatrix} a(X, Y, Z) \quad (5.12)$$

The x component of a in the (x, y, z) system is just the dot product of the x -axis, $x(X, Y, Z)$ with $a(X, Y, Z)$.

As the axes are orthogonal unit vectors, the following identity holds

$$E = CC^T \quad (5.13)$$

where C is the transformation matrix, and E is the identity matrix. The transformation matrix from (x, y, z) to (X, Y, Z) is just C^T . That is $C^T = C^{-1}$. Often it is necessary to create a transformation matrix from two vectors. The following gives an example. Two vectors are parameterized in (X, Y, Z) , N and u . Define N to be the z -axis. Then, the axes are

$$z = \frac{N}{|N|} \quad (5.14)$$

$$y = \frac{N \times u}{|N \times u|} \quad (5.15)$$

$$x = \frac{y \times z}{|y \times z|} \quad (5.16)$$

where x , y , and z are orthonormal and unit vectors.

Multiple transformations are performed by multiplying a vector by each matrix in succession.

$$u_c = C_{cb} C_{ba} u_a$$

where the notation C_{ba} is equivalent to writing $C_{a \text{ to } b}$ and the inner b frames visibly cancel out, $C_{ca} = C_{cb} C_{ba}$. An example of a sequence of rotations requiring this notation is a set of gimbals.

5.5. Quaternions

5.5.1 Introduction

A quaternion is a convenient way of representing the orientation of one frame with respect to another. Propagation of the orientation of an object with quaternions is efficient since only four elements are propagated and no trigonometric functions are needed. In addition, it does not have any singularities.

Euler angles are useful for visualizing the attitude. Quaternions should be used for numerical integration. If more than three vectors are to be transformed from one frame to another using the same quaternion, it is faster to first convert the quaternion to a rotation matrix and then do the matrix–vector multipliers to get the transformations. Transforming a vector directly with a quaternion takes 30 floating-point operations; it takes 45 operations to convert from a quaternion to a matrix. A matrix transformation takes 15 operations.

An example of a quaternion is shown in Fig. 5.4. Assume that we have two reference frames that have the same z -axis and a vector u that can be represented in either frame. The length of u is the same in each frame. In this case, the z component of u will be the same in both frames A and B but the x and y components will have different values. It is important to be precise about the definition of a quaternion. Specifically, $q_{A \text{ to } B}$

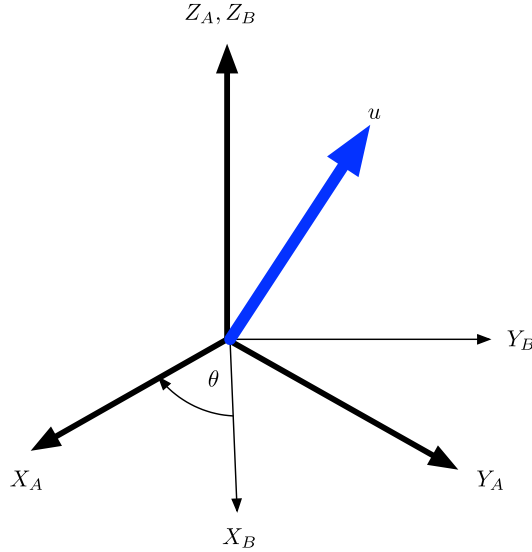


Figure 5.4 Quaternion diagram.

transforms a vector represented in the A -frame coordinates into a vector represented by the B -frame coordinates. This same quaternion might be notated instead as q_{BA} , where the order of the subscripts now indicates that the quaternion transforms a vector in frame A to a vector in frame B . This form can be helpful when combining quaternions in order as discussed below. Both are used in practice and it is important to always verify the rotation represented.

5.5.2 Fundamental properties of the quaternion

A quaternion is a four-parameter set used to describe the orientation of one reference frame with respect to a second reference frame. Although only three parameters are needed to uniquely specify the relative orientation, all three-parameter sets have singularities that make them unsuitable for numerical simulations, or integration inflight software. Four parameter sets, such as quaternions, do not have problems with singularities. The quaternion is represented by a four row-vector

$$\begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix} = \begin{bmatrix} s \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} \cos \frac{\phi}{2} \\ a_1 \sin \frac{\phi}{2} \\ a_2 \sin \frac{\phi}{2} \\ a_3 \sin \frac{\phi}{2} \end{bmatrix} = q \quad (5.17)$$

The first notation is the standard used in the software. No distinction is made between the four elements of the quaternion since numerically there is none.

Occasionally, authors will make the first element q_0 and the remainder $[q_1 \ q_2 \ q_3]^T$ to correspond to the scalar and vector notation. The second notation in (5.17) breaks the quaternion into a scalar and 3-vector part. Letting the first term be the scalar is arbitrary.

The third form relates to the theorem of Euler that any arbitrary rotation about any number of axes can be reduced to a single rotation about a fixed axis. If the rotation angle is defined as ϕ and the axis of rotation is a , then the sign of the first value is arbitrary since any rotation can be represented by two quaternions of opposite sign. In this text, the first element will always be positive. This would imply that ϕ will always be between $\pm 180^\circ$. The transpose of a quaternion is defined as

$$q^T = \begin{bmatrix} q_0 \\ -q_1 \\ -q_2 \\ -q_3 \end{bmatrix} = \begin{bmatrix} s \\ -v_1 \\ -v_2 \\ -v_3 \end{bmatrix} = \begin{bmatrix} \cos \frac{\phi}{2} \\ -a_1 \sin \frac{\phi}{2} \\ -a_2 \sin \frac{\phi}{2} \\ -a_3 \sin \frac{\phi}{2} \end{bmatrix} \quad (5.18)$$

with the property that

$$qq^T = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (5.19)$$

This notation comes from the NASA Space Shuttle world. That is, the unit quaternion has a 1 as the first element. In many organizations the unit quaternion is

$$qq^T = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad (5.20)$$

The NASA Jet Propulsion Laboratory (JPL) uses this convention. Different companies may use different conventions.

5.5.3 Quaternion nomenclature

Quaternions can be written as

$$q_{ab} \quad (5.21)$$

which is defined as the quaternion that rotates a vector from frame b to frame a (right to left in the order of the coefficients). Thus

$$u_a = q_{ab} \otimes u_b \quad (5.22)$$

and quaternion multiplication is defined as

$$q_{ac} = q_{ab} \otimes q_{bc} \quad (5.23)$$

Quaternion multiplication is defined as $\otimes$, which turns the first 4-by-1 quaternion vector into a 4-by-4 matrix that then is multiplied like any matrix.

The inner frames cancel in the multiplication. This is consistent with the notation often used for transformation matrices. Quaternion multiplication will be discussed in the next section. In the following sections, quaternions will not use the full subscript notation for compactness.

5.5.4 Quaternion operations

Expanding quaternion multiplication

$$q_1 \otimes q_2 = \begin{bmatrix} s_1 s_2 - \vec{v}_1 \vec{v}_2 \\ s_1 \vec{v}_1 + s_2 \vec{v}_2 + \vec{v}_1 \times \vec{v}_2 \end{bmatrix} \quad (5.24)$$

where s_1 is the scalar part of the first quaternion and s_2 is the scalar part of the second quaternion, $\vec{v}_1$ and $\vec{v}_2$ are the corresponding vector parts. The arrows are used to clarify that they are vectors. Quaternion multiplication can be expressed in matrix form in two ways

$$q_1 \otimes q_2 = \begin{bmatrix} q_1(1) & -q_1(2) & -q_1(3) & -q_1(4) \\ q_1(2) & q_1(1) & -q_1(4) & q_1(3) \\ q_1(3) & q_1(4) & q_1(1) & -q_1(2) \\ q_1(4) & -q_1(3) & q_1(2) & q_1(1) \end{bmatrix} q_2 \quad (5.25)$$

or

$$q_1 \otimes q_2 = \begin{bmatrix} q_2(1) & -q_2(2) & -q_2(3) & -q_2(4) \\ q_2(2) & q_2(1) & q_2(4) & -q_2(3) \\ q_2(3) & -q_2(4) & q_2(1) & q_2(2) \\ q_2(4) & q_2(3) & -q_2(2) & q_2(1) \end{bmatrix} q_1 \quad (5.26)$$

The numbers in the parentheses are the quaternion element number. The only difference between the equations is that the sign is reversed on the skew-symmetric 3×3 matrix in the lower right-hand corner. Once one quaternion is expanded into a four-by-four matrix, regular matrix operations apply.

5.5.5 Quaternion transformations

Quaternions transform vectors using the following operation

$$x_b = q_{ba} \otimes x_a \otimes q_{ba}^T \quad (5.27)$$

using quaternion multiplication with the vectors defined as quaternions with a scalar part equal to zero or

$$x_a = \begin{bmatrix} 0 \\ x_a(1) \\ x_a(2) \\ x_a(3) \end{bmatrix} \quad (5.28)$$

Writing this out in full

$$x_b = \begin{bmatrix} q_{ba}(1) & -q_{ba}(2) & -q_{ba}(3) & -q_{ba}(4) \\ q_{ba}(2) & q_{ba}(1) & q_{ba}(4) & -q_{ba}(3) \\ q_{ba}(3) & -q_{ba}(4) & q_{ba}(1) & q_{ba}(2) \\ q_{ba}(4) & q_{ba}(3) & -q_{ba}(2) & q_{ba}(1) \end{bmatrix} \begin{bmatrix} 0 & -x_a(1) & -x_a(2) & -x_a(3) \\ x_a(1) & 0 & x_a(3) & -x_a(2) \\ x_a(2) & -x_a(3) & 0 & x_a(1) \\ x_a(3) & x_a(2) & -x_a(1) & 0 \end{bmatrix} \begin{bmatrix} q_{ba}(1) \\ -q_{ba}(2) \\ -q_{ba}(3) \\ -q_{ba}(4) \end{bmatrix} \quad (5.29)$$

The middle matrix is the skew-symmetric matrix for Eq. (5.28).

5.5.6 Quaternion derivative

The derivative of a quaternion is defined as

$$\dot{q} = \lim_{\Delta t \rightarrow 0} \frac{\Delta q}{\Delta t} = \lim_{\Delta t \rightarrow 0} \frac{q(t + \Delta t) - q(t)}{\Delta t} \quad (5.30)$$

The derivative of the quaternion will be related to the angular velocity of one frame with respect to the other. There are four possible combinations for the derivative of a quaternion. They are

$$\begin{matrix} q_{ab}, \omega_a & q_{ba}, \omega_b \\ q_{ba}, \omega_a & q_{ab}, \omega_b \end{matrix} \quad (5.31)$$

The combinations are the direction of the quaternion rotation and the reference frame for the angular-rate vector. The quaternion can transform in either direction between a and b and the angular rate can be expressed either in frame a or b . Define the delta quaternion as

$$q(t + \Delta t) = \Delta q q \quad (5.32)$$

The new quaternion is just the old multiplied by a quaternion that transforms the first into the second. Write the angular rate as the product of a rotation axis, a , and a scalar angular rate

$$\vec{\omega} = a\omega \quad (5.33)$$

where a is the unit vector for $\vec{\omega}$. The derivations for each of the four sets are straightforward. Start with set q_{ba} , ω_b . Δq arises when expressed in the scalar-vector form of the quaternion,

$$\Delta q = \begin{bmatrix} \cos \frac{\omega \Delta t}{2} \\ a \sin \frac{\omega \Delta t}{2} \end{bmatrix} \quad (5.34)$$

and (5.30) becomes

$$\dot{q} = \lim_{\Delta t \rightarrow 0} \frac{\Delta q}{\Delta t} = \lim_{\Delta t \rightarrow 0} \frac{(\Delta q - e)q}{\Delta t} \quad (5.35)$$

where e is the identity quaternion

$$e = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (5.36)$$

Expressing Δq as the state-transition matrix for a quaternion it is

$$\Delta q = \begin{bmatrix} \cos(\frac{\omega \Delta t}{2}) & -a^T \sin(\frac{\omega \Delta t}{2}) \\ a \sin(\frac{\omega \Delta t}{2}) & E \cos(\frac{\omega \Delta t}{2}) - a^\times \sin(\frac{\omega \Delta t}{2}) \end{bmatrix} \quad (5.37)$$

where E is a 3-by-3 identity matrix.

$$E = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (5.38)$$

and $a^\times$ is

$$a^\times = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \quad (5.39)$$

then (5.35) becomes

$$\dot{q} = \lim_{\Delta t \rightarrow 0} \frac{1}{\Delta t} \begin{bmatrix} \cos(\frac{\omega \Delta t}{2}) - 1 & -a^T \sin(\frac{\omega \Delta t}{2}) \\ a \sin(\frac{\omega \Delta t}{2}) & E(\cos(\frac{\omega \Delta t}{2}) - 1) - a^\times \sin(\frac{\omega \Delta t}{2}) \end{bmatrix} q \quad (5.40)$$

In the limit, the cosine terms become one, and the sine terms are their argument, $\sin(x) \approx x$

$$\dot{q} = \left(\frac{\omega}{2}\right) \begin{bmatrix} 0 & -a^T \\ a & -a^\times \end{bmatrix} q \quad (5.41)$$

Using the angular vector,

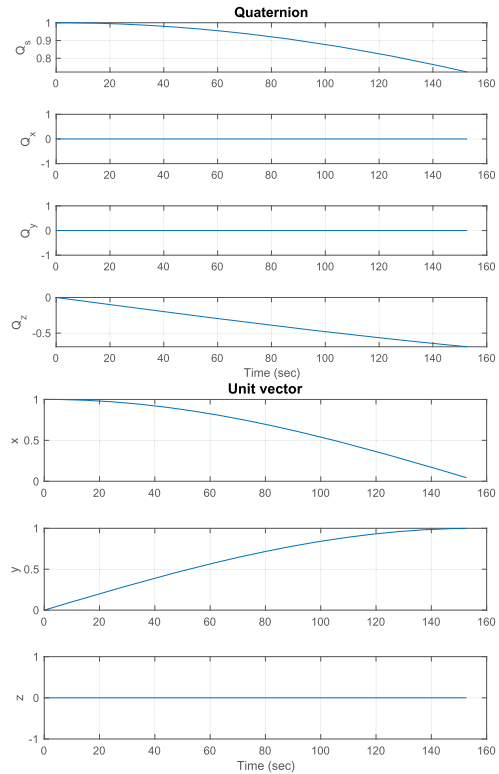
$$\dot{q} = \frac{1}{2} \begin{bmatrix} 0 & -\vec{\omega}^T \\ \vec{\omega} & -\vec{\omega}^\times \end{bmatrix} q \quad (5.42)$$

Example 5.1 shows an example of quaternion rotation. The rotation axis is chosen from the quaternion that rotates from the beginning frame to the final frame.

```

1 q1 = [1;0;0;0];
2 omega = [0;0;0.01];
3 dT = 0.1;
4 u2 = [0;1;0];
5
6 n = 3000;
7 u = zeros(3,n);
8 q(:,1) = q1;
9 u(:,1) = [1;0;0];
10 for k = 2:n
11     q(:,k) = RK4(@RHSQuaternion,q(:,k-1),dT,0,omega);
12     u(:,k) = QTForm(q(:,k),[1;0;0]);
13     if (u(2,k) > 0.999)
14         break
15     end
16 end
17
18 q = q(:,1:k);
19 u = u(:,1:k);
20
21 [t,tL] = TimeLabl((0:k-1)*dT);
22 Plot2D(t,q,tL,['Q_s' 'Q_x' 'Q_y' 'Q_z'],'
    Quaternion');
23 Plot2D(t,u,tL,['x' 'y' 'z'],'Unit_vector');
24
25 function qDot = RHSQuaternion(q,~,omega)
26
27 qDot = QToBDot(q,omega);
28
29 end

```



Example 5.1: Quaternion rotation of a unit vector.

5.5.7 Small angles

Often, control systems deal with small deviations from a nominal attitude. In these cases, all attitude representations are the same and the attitude can be treated as a vector where

the order of the rotations does not matter. A quaternion derived from small angles is

$$\begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix} \approx \begin{bmatrix} 1 \\ \frac{\theta_1}{2} \\ \frac{\theta_2}{2} \\ \frac{\theta_3}{2} \end{bmatrix} / \sqrt{1 + \frac{\theta_1^2}{4} + \frac{\theta_2^2}{4} + \frac{\theta_3^2}{4}} \quad (5.43)$$

The term in the square root normalizes the equations. Since the angles are small this term can be omitted and the small-angle quaternion becomes

$$\begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix} \approx \begin{bmatrix} 1 \\ \frac{\theta_1}{2} \\ \frac{\theta_2}{2} \\ \frac{\theta_3}{2} \end{bmatrix} \quad (5.44)$$

5.5.8 Physical interpretation of the quaternion

Since the quaternion on the first appearance does not appear to impart a physical feeling for what attitude it represents, we will go through a few examples to show that with a little effort, a quaternion is as easy to visualize as Euler angles.

5.5.8.1 Single-axis rotation

The quaternion

$$\begin{bmatrix} 0.7071 \\ 0.7071 \\ 0.0 \\ 0.0 \end{bmatrix} \quad (5.45)$$

represents a pure rotation about the x -axis. The first argument is 0.7071 and equals $\cos(90^\circ/2)$. We cannot tell the quadrant from the first argument. The second argument is the 1 component of the unit vector, (which in this case is

$$\begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \end{bmatrix} \quad (5.46)$$

times the argument $\sin(90^\circ/2)$). Since the sign is positive, the rotation must be a positive 90° rotation.

Large multiaxis rotations are somewhat more difficult to interpret.

5.5.8.2 Multiple-axis rotation

The quaternion

$$\begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} \quad (5.47)$$

represents a rotation about an axis not aligned with x , y , or z . The first argument is 0.5 and equals the $\cos(120^\circ/2)$. The $\sin(120^\circ/2)$ is 0.866. Therefore the unit vector, a , is

$$\begin{bmatrix} 0.5773 \\ 0.5773 \\ 0.5773 \end{bmatrix} \quad (5.48)$$

This quaternion represents a $+60^\circ$ rotation about a or a -60° about $-a$. If the rotation is sufficiently small, the last three components of the quaternion represent half the equivalent angles.

5.5.8.3 Small rotation

The quaternion

$$\begin{bmatrix} 0.999 \\ 0.004 \\ 0.003 \\ -0.005 \end{bmatrix} \quad (5.49)$$

represents a small rotation. The first argument equals the $\cos(0.8^\circ/2)$, which is small. Therefore the last three angles can be interpreted directly as angles (in radians). This quaternion represents approximately the rotation vector

$$\begin{bmatrix} 0.460^\circ \\ 0.344^\circ \\ 0.572^\circ \end{bmatrix} \quad (5.50)$$

5.5.9 Incremental quaternion for maneuvers

Spacecraft pointing-control systems work on small angles from the nominal orientation. A PID or PD controller works quite well. The question arises as to how large reorientations are handled. Since the definition of a quaternion is a rotation axis and an angle, large-angle maneuvers can be implemented by finding the axis and angle that reorients the spacecraft and controlling about that axis. This reduces the three axis problem to a single axis problem.

The delta quaternion is

$$q_\delta = (q^T \otimes q_t)^T \quad (5.51)$$

where q is the current quaternion and q_t is the target. Define

$$q_\delta = \begin{bmatrix} s \\ a \end{bmatrix} \quad (5.52)$$

The angle vector to the controller is

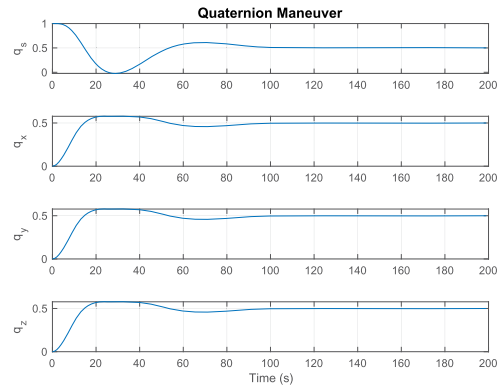
$$\theta = -2a \quad (5.53)$$

Example 5.2 shows an example of a quaternion maneuver.

```

1 n = 200;
2 xP = zeros(4,n);
3 dT = 1;
4 xPID = {[0;0] [0;0] [0;0]};
5
6 x = [1;0;0;0;0;0;0];
7 qT = [0.5;0.5;0.5;0.5];
8 accel = zeros(3,1);
9
10 [a, b, c, d] = PIDMIMO( 1, 0.7071, 0.1, 40,
    0.1, dT );
11
12 for k = 1:n
13     q = x(1:4);
14     qTB = QPose(QMult(QPose(q), qT));
15     if(qTB(1) < 0)
16         qTB = -qTB;
17     end
18     theta = -2.0*qTB(2:4);
19
20     for j = 1:3
21         xPID{j} = a*xPID{j} + b*theta(j);
22         accel(j) = -c*xPID{j} - d*theta(j);
23     end
24     xP(:,k) = q;
25     x = RK4(@RHS,x,dT,0, accel);
26 end
27
28 t = (0:n-1)*dT;
29 yL = {'q_s' 'q_x' 'q_y' 'q_z'};
30 Plot2D(t,xP,'Time(s)',yL,'Quaternion_Maneuver');
31
32 %% Right hand side [angle;rate]
33 function xDot = RHS(x,~, accel)
34
35 q = x(1:4);
36 omega = x(5:7);
37 qDot = QToBDot(q, omega);
38 xDot = [qDot; accel];
39
40 end

```



Example 5.2: Quaternion maneuver. The spacecraft model is three double integrators.

5.5.10 Angle and unit vector to a quaternion

If you have an angle, θ , and a unit vector u you can create a quaternion

$$q = \begin{bmatrix} \cos \frac{\theta}{2} \\ u \sin \frac{\theta}{2} \end{bmatrix} \quad (5.54)$$

This is already normalized to one.

5.5.11 Axis-alignment quaternion

A frequent need is to align an axis in the spacecraft with an inertial axis. The Hubble Space Telescope does this whenever it begins an observation.

Take two unit vectors, u , and v . u is fixed to the spacecraft. v is in the inertial frame. First, find the dot product of the two vectors

$$d = u^T v \quad (5.55)$$

If $d = -1$ we have a special case. Otherwise, the scalar part of the quaternion is

$$s = \sqrt{2(1 + d)} \quad (5.56)$$

The crossproduct of the vectors is the axis of rotation

$$c = u \times v \quad (5.57)$$

The final quaternion is

$$q = \begin{bmatrix} \frac{1}{2}s \\ \frac{c}{s} \end{bmatrix} \quad (5.58)$$

This should be normalized.

5.5.12 Small angles

Often satellites are planet pointing. This means that they rotate at orbit rate about one axis. The planet pointing frame is often local vertical/local horizontal (LVLH). If the angles are small, the transformation matrix from LVLH to the body is

$$A = \begin{bmatrix} 1 & \theta_z & -\theta_y \\ -\theta_z & 1 & \theta_x \\ \theta_y & -\theta_x & 1 \end{bmatrix} \quad (5.59)$$

with θ_x about x , θ_y about y and θ_z about z . The angles and signs can be found by inspection by looking at Fig. 5.5.

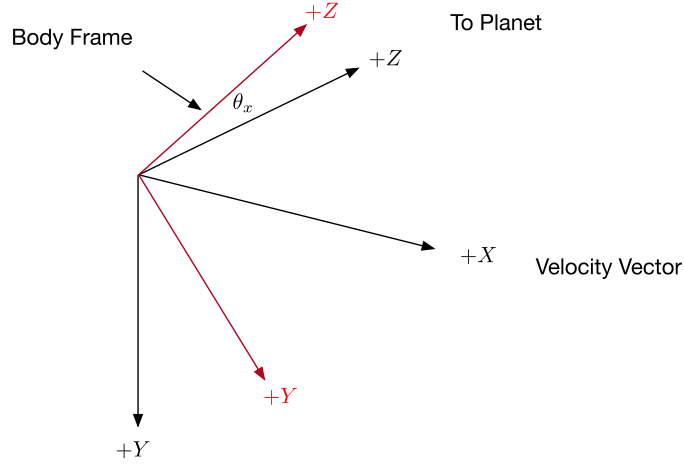


Figure 5.5 Coordinate transformations from LVLH to body with small angles. This diagram shows the roll angle, α . The body axes are in red.

Linearize the matrices so that $\cos \theta = 1$ and $\sin \theta = \theta$.

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & \theta_x \\ 0 & -\theta_x & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -\theta_y \\ 0 & 1 & 0 \\ \theta_y & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & \theta_z & 0 \\ -\theta_z & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (5.60)$$

Multiplying the matrices and dropping products of angles

$$M = \begin{bmatrix} 1 & \theta_z & -\theta_y \\ -\theta_z & 1 & \theta_x \\ \theta_y & -\theta_x & 1 \end{bmatrix} \quad (5.61)$$

which is the same as Eq. (5.59).

5.5.13 Quaternion interpolation

Sometimes, it is desired to interpolate between two quaternions without doing quaternion propagation. The SLERP algorithm provides a way of interpolation. Assume there are two quaternions q_1 and q_2 . Find their dot product

$$c = q_0^T q_1 \quad (5.62)$$

Let t be an array of the intermediate values for the quaternion. This ranges from 0 to 1. If c is less than zero, flip its sign and flip the sign of q_1 . If c is far from 1 compute

$$\omega = \cos^{-1} c \quad (5.63)$$

$$s = \sin \omega \quad (5.64)$$

$$s_0 = \frac{\omega}{s} \sin(1 - t) \quad (5.65)$$

$$s_1 = \frac{\sin(t\omega)}{s} \quad (5.66)$$

If ϵ is near one, let $s_0 = 1 - t$ and $s_1 = t$. The interpolation is

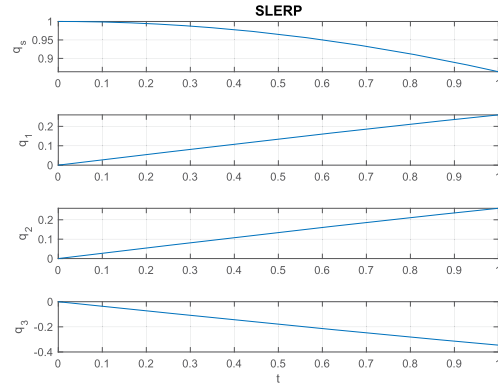
$$q = s_0 q_0 + s_1 q_1 \quad (5.67)$$

This is often used in computer graphics.

Example 5.3 shows an example of quaternion rotation. The rotation axis is chosen from the quaternion that rotates from the beginning frame to the final frame.

```

1 q1 = QUnit([1;0.3;0.3;-0.4]);
2 q0 = [1;0;0;0];
3 t = linspace(0,1);
4
5 q = QSLERP( q0, q1, t );
6
7 yL = {'q_s' 'q_1' 'q_2' 'q_3'};
8
9 Plot2D(t,q,'t',yL,'SLERP')
```



Example 5.3: Quaternion interpolation.